



DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Development of a Network Intrusion Detection Demonstrator for Industrial Control Systems

Fabian Pröbstle





DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Development of a Network Intrusion Detection Demonstrator for Industrial Control Systems

Entwicklung eines Network Intrusion Detection Systems für industrielle Kontrollsysteme

Author:	Fabian Pröbstle
Supervisor:	Prof. Dr. Claudia Eckert
Advisor:	Mohammad Reza Norouzian
Submission Date:	15.09.21



I confirm that this bachelor's thesis in informatics is my own work and I have documented all sources and material used.

Munich, 15.09.21

Fabian Pröbstle

Acknowledgments

I want to thank my advisor Mohammad Reza Norouzian. I'm very grateful for your support and feedback. I also want to thank my family and friends who were always there for me.

Abstract

Industrial control systems are used in vital areas such as energy supply and production. The security of industrial control systems is an ever-emerging issue. Various attacks on such control centers have already caused huge damage. Early detection is the first step to do something against an attack. Intrusion Detection Systems are used to detect suspicious network traffic or activities. We would like to apply this technology in the area of industrial control systems to demonstrate its usefulness. For this purpose, we extend a virtual control system testbed with additional attack methods. Several common attack detection tools will be compared and finally Snort will be used to detect the implemented attacks. This demonstration can be used to enable attack detection in critical systems. In addition, attack detection techniques themselves can also be further improved.

Zusammenfassung

In lebensnotwendigen Bereichen wie der Energieversorgung und Produktion werden industrielle Kontrollsysteme eingesetzt. Die Sicherheit von industriellen Kontrollsystemen ist ein immer wieder aufkommendes Thema. Diverse Angriffe auf solche Kontrollzentren haben bereits riesige Schäden angerichtet. Die frühzeitige Erkennung ist der erste Schritt um etwas gegen einen Angriff zu unternehmen. Intrusion Detection Systems (Angriffserkennungssysteme) werden dazu genutzt, verdächtigen Netzwerkverkehr oder Aktivitäten zu erkennen. Diese Technologie möchten wir im Bereich der industriellen Kontrollsysteme einsetzen und dadurch ihren Nutzen demonstrieren. Dafür erweitern wir ein virtuelles Kontrollsystem-Testbed um weitere Angriffsmethoden. Mehrere gängige Angriffserkennung Werkzeuge werden verglichen und zum Schluss Snort dazu genutzt, die implementierten Angriffe zu erkennen. Diese Demonstration kann dazu genutzt werden, Angriffserkennung in wichtigen Systemen zu ermöglichen. Außerdem können auch die Angriffserkennungstechniken an sich weiter verbessert werden.

Contents

Acknowledgments	iii
Abstract	iv
Zusammenfassung	v
1 Introduction	1
1.1 Motivation	1
1.2 Research Question	1
1.3 Structure of the thesis	2
2 Foundations	3
2.1 Industrial Control Systems	3
2.1.1 SCADA or DCS	3
2.1.2 PLC based Topologies	4
2.1.3 Ladder Logic	4
2.1.4 Modbus	4
2.2 Intrusion Detection Systems	5
2.2.1 Detection Methods	7
2.2.2 Types of Intrusion Detection Systems	7
3 Related Work	9
3.1 Modbus attack frameworks	9
3.2 Other countermeasures against SCADA attacks	10
3.3 Scada Testbeds	10
4 Testbed	12
4.1 Components	12
4.1.1 Programmable Logic Controller	12
4.1.2 Historian	12
4.1.3 Human Machine Interface	12
4.1.4 Process Simulator	13
4.1.5 Visualizer	13

Contents

4.1.6	Attacker Machine	13
4.2	Multistage extension	13
4.2.1	Cooker	13
4.2.2	Spice	13
4.2.3	Milk	14
4.2.4	Pumps	14
4.3	Industrial Process Simulation	14
4.4	Historian	17
4.5	Human Machine Interface	17
4.6	Attacks	18
4.6.1	Basic Functions	18
4.6.2	Request flood	18
4.6.3	Syn- flood	18
4.6.4	Multi-Stage MitM attack	19
5	Intrusion Detection	21
5.1	NIDS Comparison	21
5.2	Snort3	23
5.2.1	Rules	23
5.3	Attack monitoring	24
5.3.1	Monitoring of unauthenticated requests	24
5.3.2	Monitoring of flooding attacks	24
5.3.3	Monitoring of the multistage attack	25
6	Summary and Conclusion	26
6.1	Limitations	26
6.2	Future Work	27
	List of Figures	28
	List of Tables	29
	Bibliography	30

1 Introduction

1.1 Motivation

Industrial Control Systems(ICS) are used in various areas. Their task is to provide a simple and safe mechanism to control a real physical process. A few areas where Industrial Control Systems are used are factory automation, power plant control and supervision of chemical processes. Industrial Control Systems are also affected by the growing interconnection of technical devices via the internet. Where there used to be isolated networks, there are now more and more connections to the outside. This creates a security risk. A report from Kaspersky notes that over 38% of all ICS computers were targeted by attacks in the year 2020.[Kas21] Attacks on Industrial Control Systems have created large damage in the past.[H+18] Popular examples are Stuxnet(2010), Black Energy(2014), and Triton(2017). All of those attacks have in common that they were specifically designed to attack Industrial Control Systems. To develop such a tool a lot of money is invested. The source code of Stuxnet included four previously unknown exploits, which are rare and can be sold for high prices. Stuxnet and Black Energys main targets were of economical and political nature. Stuxnet damaged one-fifth of the nuclear centrifuges in Iran. Black Energy set, among other things, parts of the Ukrainian power grid offline for up to 6 hours. Tritons objective, on the other hand, was to infect industrial safety systems by the company Schneider Electrics. A malfunction has dangerous consequences for humans. This shows clearly that Industrial Control Systems are an attractive target and therefore more work has to be done to protect these systems.

1.2 Research Question

In this thesis we concentrate on how to detect attacks on ICS trough the analysis of network traffic. Furthermore, we want to create a demonstration of how network based attack detection can be applied to ICS. This is why we asked our self the following two research questions:

- How can we detect attacks on an ICS through network analysis?

- How to develop a demonstration of a Network Intrusion Detection System applied in the context of Industrial Control Systems?

1.3 Structure of the thesis

The work of this thesis is composed of two elements which are supposed to give an answer to one research question respectively. To provide a foundation Chapter 2 illustrates some of the technologies used in the rest of the thesis. Section 2.1 focuses on the basic functionality of Industrial Control Systems and the structure of the Modbus protocol. Section 2.2 deals with Intrusion Detection Systems in general. The main part of this thesis start with Chapter 3, where the basis for the detection demonstrator is laid. Therefore, a testbed is used that emulates a Industrial Control System. Section 4.1-4.5 explain the changes that were made to the testbed. Section 4.6 describes attacks implemented for later detection. In the Chapter 5 concentrates on the first research question and how we can apply network-based intrusion detection. In Section 5.1 multiple detection systems are compared. The rest of the chapter is addressed on how attacks can be detected with the help of Snort. Chapter 6 concludes the whole thesis, discusses its limitations and expresses ideas for future works.

2 Foundations

2.1 Industrial Control Systems

An industrial control system (ICS) is a broad class of automation systems used to provide control and monitoring functionality in manufacturing and industrial facilities.[KL14] An average ICS is composed of a control center, a human machine interface, a historian, multiple controllers that regulate and monitor different actuators and sensors.[S+11] The control center manages the whole process plant. It gives directions to the smaller control units. A Human Machine Interface (HMI) is a device used to view the process of the automated task and manually interfere if necessary. It is the bridge between the human and the process plant. The historian collects data of processes and sensors and stores them. This data can be used to comprehend past automatized decisions. Controllers are typically either a Programmable Logic Controller (PLC) or a Remote Terminal Unit (RTU). Both have the job to collect input data and relay them somewhere else. A RTUs main task is “transport” therefore they often have integrated communication technology, such as cellular data. A PLC on the other side is used to control processes. It is capable of rapid decision making. This distinction is not always that clearly defined in a modern environment. A PLC can handle the tasks of a RTU and the other way around. Both types of control units can also be used next to each other in an ICS. A sensor detects environment changes and sends them to a control unit. A few examples are temperature, humidity, velocity, torque,etc. An actuator performs an operation with an effect on the real world. Examples for actuators are valves, pumps, relays, etc.

2.1.1 SCADA or DCS

Supervisory Control and Data Acquisition (SCADA) and Decentralised Control Systems (DCS) are two types of ICS. Today the differences between both types is faint.[KL14] One historical difference was that the control and process location in a DCS are geographically located near each other. In a SCADA system the components can be separated.

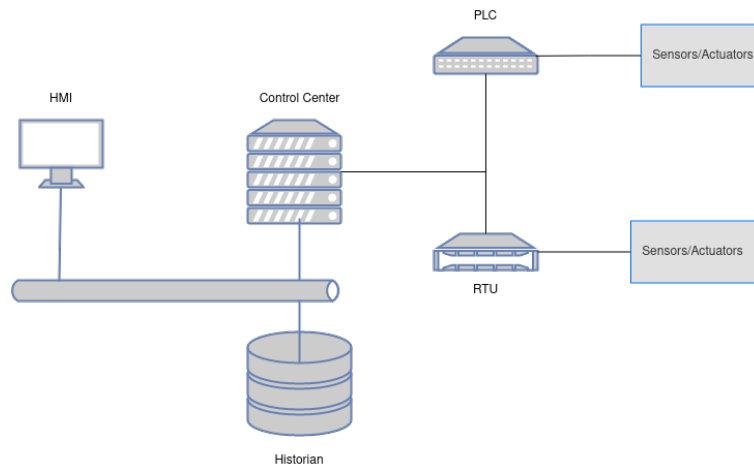


Figure 2.1: Example of an ICS

2.1.2 PLC based Topologies

It is also possible to construct a simple ICS without the use of a control center. One or multiple PLCs adopt the control role. Collaboration between the PLCs is harder, due to the lack of a supervisor device.[S+11]

2.1.3 Ladder Logic

Programmable Logic Controllers use Ladder Logic, a simple programming language. Ladder logic originates from physical electronic relays. When a PLC executes the ladder logic it follows a thought wire until it reaches an input relay. If the relay is open (input == true) the PLC continues to follow the wire, otherwise it starts at the beginning again. If the PLC reaches a coil at the end of such a wire, it sets the output to true. Ladder Logic can also handle non-binary inputs they can be compared with other values through logical gates such as "Greater than"(GT).

2.1.4 Modbus

Modbus is a communication protocol located at OSI-Layer 7, that is used in the context of industrial automation. Since its emergence in 1979 it was and is still used in many applications. It works with a standard Client/Server Approach who are in the Modbus syntax also called Master/Slave. The Master sends a Modbus-request to a slave, which handles the request and sends the relevant data in a response.

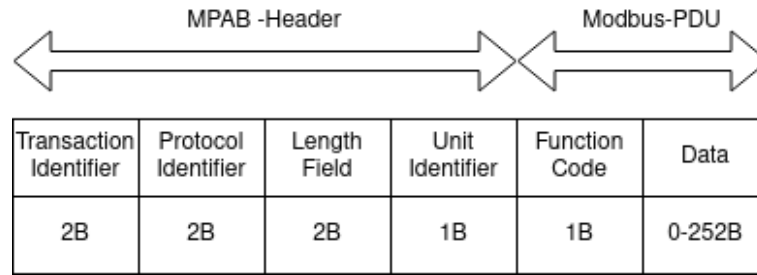


Figure 2.2: Modbus/TCP Packet

Modbus/TCP

Modbus/TCP is a protocol that can be used for the communication between all components of an ICS. It is an extension of the Modbus Protocol that allows transmission of TCP packets instead of serial communication. Fig. 2.2 shows the structure of a Modbus/TCP packet. It consists of the an MBAP header and the Modbus PDU, which is the same as in Serial Modbus packets. The Transaction Id is used to match responses and requests. The protocol Identifier 0 classifies the Modbus protocol. Other numbers can be used for intra-system multiplexing. The packet length is encoded in the length field. The unit identifier contains the slave id to make them distinguishable from the master. When talk about Modbus in the following we always mean Modbus/TCP.

The function code in a Modbus packet defines an action that the recipient should execute. Individual functions can be supported by different PLCs hardware. Table 2.1 shows common functions. Stored data is put in one of four tables on a Modbus device. Discrete Inputs and Input Registers have the Write and Read Permission, while Coils and Holding Registers are only readable. Discrete Inputs and Coils can store only 1 bit. Registers store one 16-bit word. (Table. 2.2)

2.2 Intrusion Detection Systems

The task of an intrusion detection system (IDS) is to monitor, analyze and report events that happen in a computer system or network.[SM07] These events can occur through malware, attackers or accidentally. It is not the responsibility of an IDS to avert an attack. That's why intrusion detection systems are complemented by intrusion prevention systems(IPS). Which take the reports from an IDS and are able to perform counter measurements against the incident. IDS can also be useful to document or gather more information on on-going attacks.

Function-Code	Function
01	Read Coils
02	Read Discrete Inputs
03	Read Holding Registers
04	Read Input Register
05	Write Single Coil
06	Write Single Register
15	Write Multiple Coils
16	Write Multiple Registers
23	Read/Write Multiple Registers
43	Read device Identification

Table 2.1: Modbus-Function Codes

Discrete Input	Coil	1 bit
Input Register	Holding Register	16 bit word
Read/Write	Only Read	

Table 2.2: Modbus Data Modeling

2.2.1 Detection Methods

Two types of detection methods are signature based detection and anomaly based detection.

Signature-based Detection

For signature-based detection the signatures of network traffic, files or programs are compared with an intrusion pattern. If a signature fits into a pattern it gets reported. Signature-based detection is very effective in detecting previously known attacks. If the signature of an attack is completely new or altered, this method is unable to detect the attack. Signature-based detection is also incapable of getting a deeper understanding of bigger processes. If an attack happens in multiple smaller steps, where each step does not contravene against a rule, the signature based approach might not detect the attack.

Anomaly-based Detection

The anomaly-based approach is to learn how a network or computer works and communicates in a normal environment. Therefore, the detection system generates activity profiles during a training period. After the training period all events are compared with those profiles and deviations are reported. The profiles can be either static or dynamic. A static profile does not change, while a dynamic profile learns new behaviours of a computer system after the training period. An anomaly based detection system is powerful in finding previously unknown threats due to its white-list like detection capabilities. One disadvantage of an anomaly based system is that non-threatening event that happens very rarely, like a monthly backup, can raise an alarm. Another disadvantage is that attacks that happen very frequently could be learned as normal behaviour and aren't detected anymore.

2.2.2 Types of Intrusion Detection Systems

Two ways to monitor computer systems are Host-based and Network-based Systems

Host-based Systems

A host-based IDS monitors all relevant information for one specific host. This can be incoming/outgoing traffic, processes, log files or configuration changes. Host-based IDS are mainly installed on important servers. For some devices like micro-controller based PLCs this technique may be unsuited due to the difficulty of installing the detection system on the host.

Network-based Systems

A network-based IDS(NIDS) monitors the network traffic at one or multiple locations. The detection devices are often installed at the boundaries between two networks. For the detection of multiple devices, this kind of IDS is easier to setup. They are unable to detect attacks that infect a device over other ways than the network, for example by manually inserting a USB stick.

3 Related Work

3.1 Modbus attack frameworks

The main focus of this thesis lays on detection, but therefore it is also important to have attacks that can be detected. Smold is a modular framework that can be used for penetration testing in Modbus environments.[16] It is able to send Modbus read and write requests to a client. This functionality is called fuzzing, but offers no ways of automatically testing different input values on a large scale. Every write request has to be setup manually. It is no real fuzzing. The framework can also check whether a Modbus device is running on a specific IP address and port. Therefore it tries to open a SocketStream and if the connection is established it reports that a Modbus device is running. This method is not very reliable, other TCP services except Modbus could also accept the connection. Another module is the Modbus function scanner, which brute forces all function codes and returns a list of all that didn't return an exception. The last module worth mentioning is the ARP-poisoning attack. This module uses ARP-spoofing to direct all Modbus traffic through the attacker device. We noticed that the whole framework is not very reliable and the python 2.7 code is outdated. Nonetheless does this framework provide a basic repertoire of attacks. The famous Metasploit framework also provides a module called Modbus Utility Client.[18a] This client provides similar functions as the Smold framework. It can also send all kinds of read and write requests. It is easier to use and more reliable. Another Metasploit module called Modbus Version Scanner can also detect devices that use the Modbus protocol.[18b] In contrast to Smold, a read request is sent to a specified IP address and if a valid response can be received a Modbus device is reported.

3.2 Other countermeasures against SCADA attacks

Many factors play a role when it comes to security. This thesis wants to improve ICS security through intrusion detection system, but there are also other possibilities. A Swedish team of researchers examined the efficiency of such methods.[Kor+17] For the evaluation they looked at the estimated distribution of time different populations of attackers need for a successful attack on a single SCADA component. Granular network segmentation was the most successful approach. Followed by keeping all systems up-to-date and enforcement of network access through port security, static ARP tables, and password quality. All methods sound basic, but showed great effects on the time an attacker needs.

The procedures used by the team and the approach of intrusion detection are both effective tools to prevent infiltration and harm to control systems. How they do it differs greatly. The methods mentioned here are like a wall that should hinder an attacker from damaging the system, while intrusion detection, which is used in this thesis, works like an alarm bell that gives alerts when an attack is happening. The best security is only achieved when both varieties work together.

3.3 Scada Testbeds

One of our research questions is how to create a NIDS demonstrator. A requirement is to have a testbed on which attack can be applied. The Secure Water Treatment Testbed by iTrust Singapore is a real physical water treatment plant. It is equipped with modern SCADA devices and state of the art water processing gear. In this testbed was a hierarchical intrusion detection system(HAMIDS) integrated.[GT16] The networks are separated into multiple demilitarized zones which makes a hierarchical IDS necessary. Multiple hackers were invited to attack the water treatment plant and the detection capabilities of HAMIDS were tested. There are also datasets available that contain the network traffic of the plant under normal circumstances and under different attacks.[21a] On a hardware testbed you can test all possible types of attack without destroying critical infrastructure. The downside is that such a testbed is expensive and located at one fixed place. The goal of this thesis is to create a similar demonstration of an IDS, but on an emulated Testbed. Such a testbed was created in the thesis of Victor Embacher. [Emb21] The version described in the thesis is based on a network of virtual machines that simulate an ICS. Therefore, it is portable and very easy to setup. It is not limited to only one kind of industrial process and can be adapted to support different kinds of attacks. The used communication protocol is Modbus. The testbed is continuously updated and now there is also a version that utilized docker-containers.

All devices communicate over a bridge interface. There are now demilitarized zones as of now. The testbed also comes with a MitM-attack. Trough ARP-poisoning are all Modbus packets redirected over the attacker and modified.

4 Testbed

Our second research question is how to develop a demonstration of a demonstration of a Network Intrusion System applied in the context of Industrial Control Systems? We decided to base this work on a testbed[Emb21] for Industrial Control Systems created by Victor Embacher.

4.1 Components

The testbed emulates a SCADA environment through a docker container network. It consists of the following 6 components:

4.1.1 Programmable Logic Controller

A component that simulates a hardware PLC. OpenPLC is used here it can be configured using a web interface and executes Ladder Logic Programs.

4.1.2 Historian

In the original Testbed the TICK-Stack from Influx was used as a historian. TICK stands for Telegraf, InfluxDB, Chronograf and Kapacitor. It was developed by Influx. Telegraf is a data gathering tool. The testbed utilizes it to collect the Modbus network traffic and redirect it to Influxdb. Influxdb is a database specialized on timeseries data. Chronograf is the corresponding web interface that is capable of showing dashboards and generating alerts. Kapacitor is used to process data sent to InfluxDB and transform it to the users needs.

4.1.3 Human Machine Interface

Fuxa is a tool for web based process visualization. It comes with an editor that allows to create and change views. The information shown in the views comes directly from Modbus traffic.

4.1.4 Process Simulator

The process simulator is a python program that simulates a water heating process. The resulting temperature and pressures values are published to the PLC over Modbus slaves.

4.1.5 Visualizer

The visualizer is a javascript web app that is used to visualize the process. It more or less duplicates the information from the Chronograph dashboard and Fuxa-interface. It has no use for our extension and use-case.

4.1.6 Attacker Machine

The attacker machine simulates the computer that an attacker would use to launch further attacks after infiltrating the network. It runs a light version of Kali pentesting OS with no tools pre-installed. It contains python programs to execute a Man in the Middle-Attack on the SCADA-System, where temperature and pressure limits are raised to damage the process.

4.2 Multistage extension

We extended the testbed by a process that includes a total of four PLCs. With this new version we wanted test detection of multistage attacks. Only small variations are made in each PLC control loop to achieve a grave change of the process outcome.

For our purpose it is not important that this process is a simulation of a real physical system. Therefore, we decided to implement an abstraction of cooking a chai late tea. This procedure can be split into the following four tasks, each of which is controlled by its own PLC.

4.2.1 Cooker

If the water-level in a tank reaches a minimum value(level_sensor) and the module is turned on(on_button), a heating element(cooker_on) starts to operate until the water temperature reaches the maximum value of 120°C(max_temp). (Fig. 4.1)

4.2.2 Spice

If the water-level(level_sensor) in a tank reaches a minimum value and the module is turned on(on_button), five spice called "S", "P", "I", "C", "E" are added to the tank

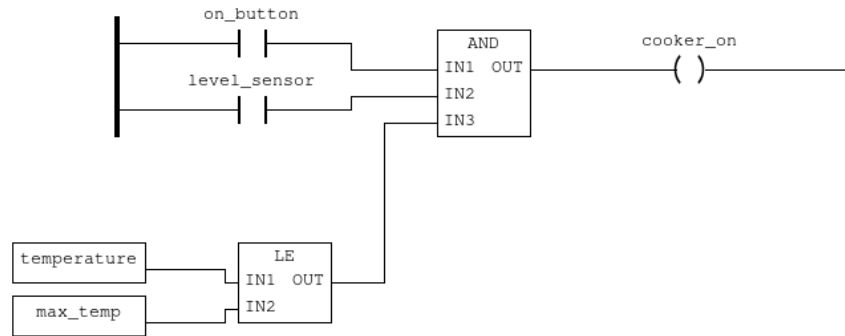


Figure 4.1: Ladder logic cook PLC

(different pumps) with different supply rates, until their maximum concentration is reached.(Fig. 4.2)

4.2.3 Milk

If the water-level in a tank reaches a minimum value(level_sensor) and the module is turned on(on_button), milk is added to the tank until the tea reaches a drinkable temperature of 65°C (min_temp) (Fig. 4.3).

4.2.4 Pumps

The pump PLC supervises all the inlet and outlet flows of the processing tanks. It empties one tank until the fill level drops under a minimum value(level_sensor), the maximum fill value of the next tank is not reached and the next tank is not emptied at the moment. It is also checked whether a processing step is finished, which means that a temperature or concentration limit is reached. The total concentration limit is 52. For the cook and milk steps the outflow already starts at 100°C and 70°C. If all conditions are met the different pumps are activated. (Fig. 4.4)

4.3 Industrial Process Simulation

The industrial process simulation had to be adapted to the use of multiple PLCs. Four python classes were created that contain the necessary simulation functions. Sensor information from a single tank is used in multiple PLCs. For example is the cooking temperature used to regulate the heater and to decide when the process has finished and

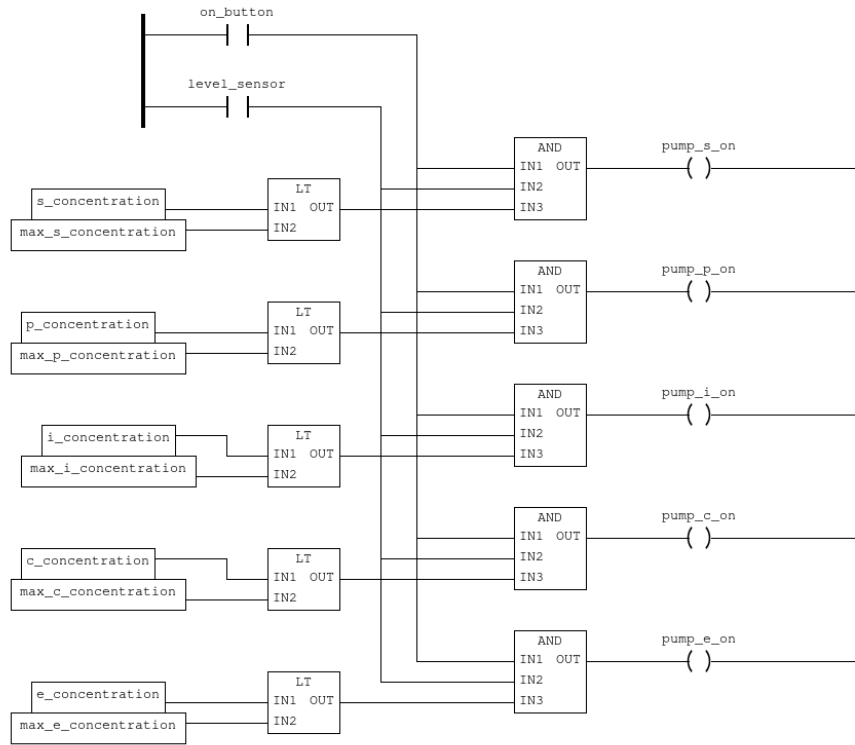


Figure 4.2: Ladder logic spice PLC

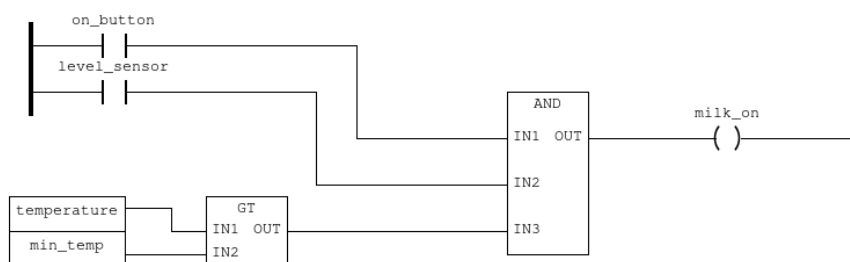


Figure 4.3: Ladder logic milk PLC

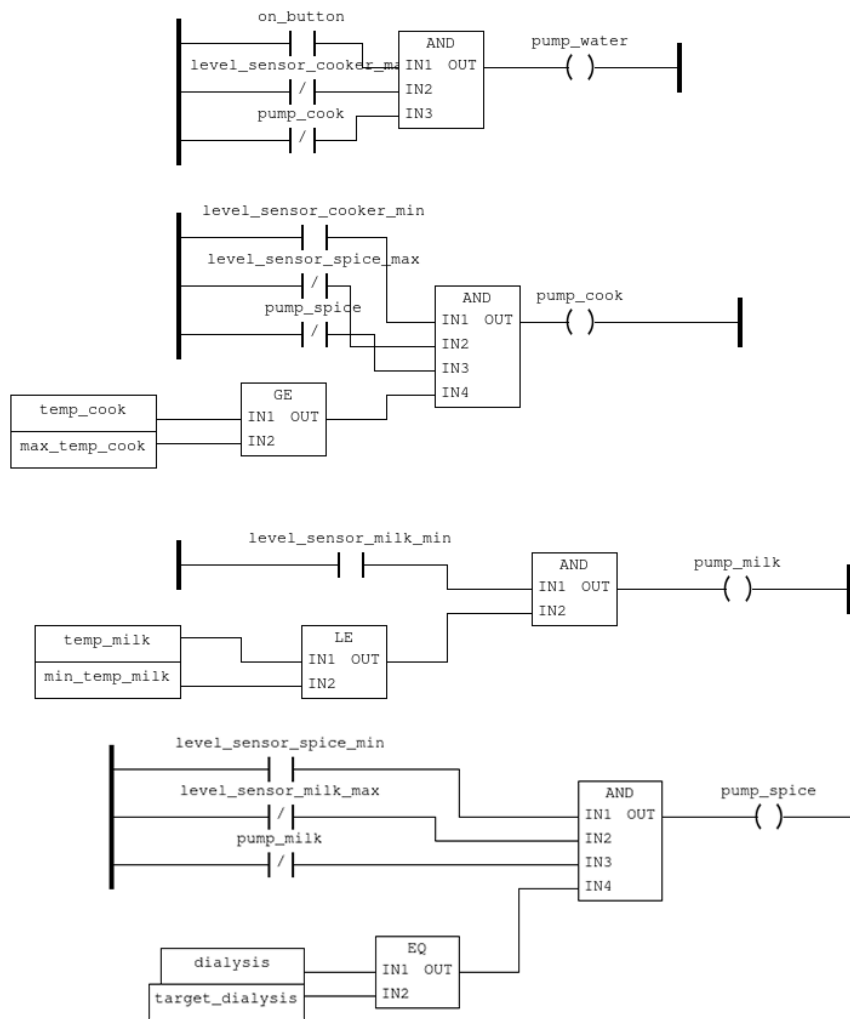


Figure 4.4: Ladder logic pump PLC

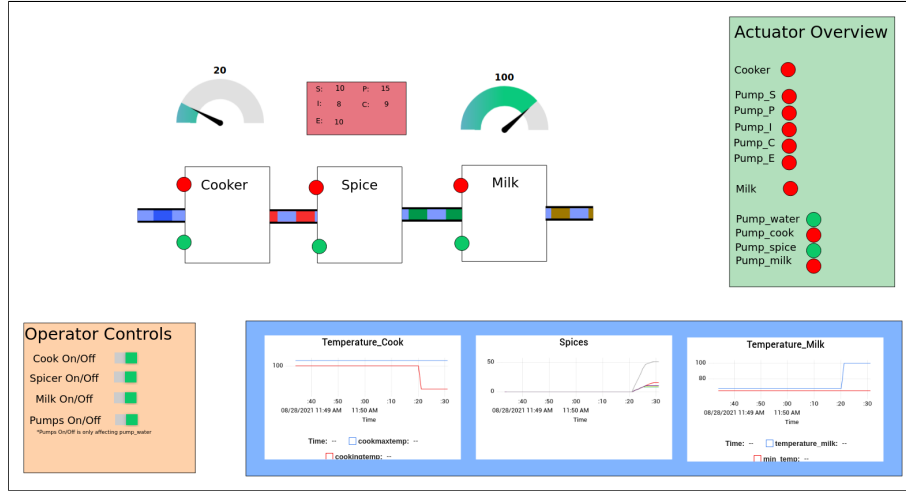


Figure 4.5: HMI view

the pump can be activated. The ability of threads to share global variables made them easier to use than single CPU processes. That's why all simulators are implemented using threads. The Modbus slaves are using process and use the ports 5020-5023.

4.4 Historian

The TICK-Stack offers an easy to use database and dashboard. It works really well with numbers and can also show strings. It is however not that practical for parsing and showing logfiles of attack logs. For the implementation of a NIDS this feature is required. That's why the historian was changed to the elastic-stack. We still use Telegraf from the tick stack to send the Modbus traffic to Elasticsearch. The results are now displayed by Kibana instead of Chronograf.

4.5 Human Machine Interface

The HMI also had to be adapted. Figure 4.5 shows a picture of the new view. In the center are the three tanks with temperature and spice gauges above them. The fill level is indicated by the red/green dots next to the tanks. On the left is an actuator view that shows which actuators are activated. On the bottom left is a control panel to de-/activate the different PLCs. The bottom right show diagrams of the two temperatures and the spice values.

4.6 Attacks

The following attacks were implemented in order to test the detection in the next chapter.

4.6.1 Basic Functions

Modbus is an unauthorized protocol, which means that every entity in a network can send requests to another Modbus device. The other device can't know if the request is legal or comes with malicious intent. An attacker can use basic Modbus requests to read and write registers on the corresponding slave. That is why this is a very simple but also highly effective way of attacking a Modbus network. Helper methods were added that make those requests available in one line python functions. Their syntax is based on the one used in the Metasploit auxiliary tool modbusclient.[18a] The following functions are implemented:

- read coils(normal/discrete input)
- read registers (holding/input)
- write to coils
- write to registers

4.6.2 Request flood

A request flood can be used to achieve a denial of service(DOS) by sending too many requests to a Modbus device. A method was implemented that opens multiple threads which send random write requests to the attacked device. Too many requests mean that not every single one can be processed. The Modbus network can't work as it should under normal conditions.

4.6.3 Syn- flood

A SYN flood attack abuses the nature of the transmission control protocol(TCP) to achieve a denial of service. To make sure that a lost packet can be resent TCP uses sequence numbers. They are shared in three-way handshake. The client sends a random sequence number to the server. The server acknowledges the sequence number and in the step response with its own sequence number. The client acknowledges the sequence number of the server.

During a SYN flood many SYN packets are sent out. The incoming SYN-ACK packets are ignored and no ACK are sent back to the server(dashed line). The attacked device

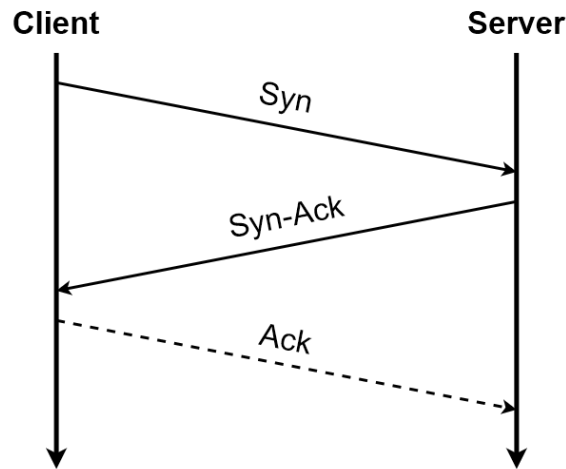


Figure 4.6: TCP - Handshake

stores information for every received Syn request. If it receives too many in a short amount of time, it will run out of storage and new devices can't connect anymore.

4.6.4 Multi-Stage MitM attack

The multistage attack utilizes a reconnaissance phase and an attack phase to find open Modbus ports and launch an attack accordingly.

Reconnaissance

Modbus function code 0x43 is reserved for device information. When an attacker sends a request with this code to an open port, it will receive a valid response. Through the function code 0x43 we will receive information about the used hardware, which can be useful to find exploits. OpenPLC doesn't support this function code. We still get a response with an exception. The exception is also a proof for a Modbus device. By sending a Modbus device information request to port 502 of every IP address in a specified range, we can create a list of devices that use the Modbus protocol. For a more deeper search it is also possible to loop over all port addresses. This technique can be used to find Modbus devices that do not use the standardized port 502. The next step is to gather information about all used registers. A function reads out the holding registers of the selected device over a specified time period. The register values are displayed on the terminal. At the end of the specified time period all registers that are not equal to zero and all that changed in this time period are highlighted.

Attack

We use a version of the MitM-attack that comes with the testbed. ARP-cache of master and slave are poisoned with the attackers mac-address. It now acts as a man in the middle. Modbus lack of encryption makes it easy to modify packets and change the outcome of the process. Our attack changes the gradient angle of all temperature curves. The cooker PLC gets an input of 100°C while the actual temperature reported by the thermometers is only 70°C. The outcome of our industrial process would change in real life, because the water is not boiled correctly and therefore the spices won't dissolve in the water as they should. This procedure is similar to the one used in the Stuxnet attack, where the acceleration patterns of the nuclear centrifuges were changed. Plant operators have a hard time noticing the changes, because the dashboards and human machine interfaces will show the altered values.

5 Intrusion Detection

5.1 NIDS Comparison

There are multiple Open Source Network Intrusion Detection Systems available. We examined Snort3, Suricata and Zeek closer. In table 5.1 are the most relevant aspects for our use-case listed. Snort3 is the new version of Snort released in the beginning of 2021. It includes all capabilities of the old Snort but is more efficient and adaptable. It uses its own rule format to specify which events trigger an alarm. Suricata uses similar rules, but also includes a script based approach. This offers more complex ways of detecting threads. It also capable of detecting protocols without the use of specific ports. Zeek (formerly Bro) uses its own scripting language to make alarm decisions. This language is more complex than the rule based approach, but also gives more freedom. Another feature is the so-called Zeek-Cluster.[21c] It allows to setup and configure nodes on multiple computers. The nodes can function as a worker, logger, manager, proxy. The worker nodes sniff the network traffic and perform the packet analysis. The logger stores all log files. This function can also be undertaken by the manager node. The manager is used to collect the logs and perform high level processing. Such a cluster architecture can be useful in bigger networks. Through the global high-level view of the manager nodes it can monitor multiple isolated networks and detect global multistage changes in the behaviour of the ICS.

Modbus is famous for its simplicity and in our testbed all used ports are known beforehand. The testbed also mimics the production end of an ICS the communication works over a docker-bridge and there are no separated networks. Suricatas scripting and protocol detection features have no big impact on the result. The same goes for Zeeks scripting and clusters features. That's why we decided that Snort3 with its simple and efficient design is the right NIDS for this project.

	Advantages	Disadvantages
Snort3	• Many rules included • Its easy to create new rules • Supported by Cisco	• No cluster support
Suricata	• Can find protocols without specific ports • Supports rule definitions and scripting • Many rules included • JSON Output	
Zeek	• Allows to setup clusters to collect traffic from multiple network locations. This traffic is monitored on a single manager node. This can help to examine the propagation of a malware in your network. • Zeeks own turing complete scripting language provides more possibilities than simple rules.	• All nodes have to use the same OS-Version • All nodes need to have the same user with promiscuous network interface mode rights. • The scripting language makes it more complicated to implement rules

Table 5.1: Comparison of different NIDS-Solutions

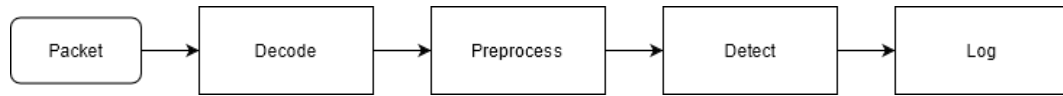


Figure 5.1: Snort detection flow

5.2 Snort3

Snort3 is the newest version of the open-source network intrusion detection system Snort.[Sno21] To detect attack in network traffic multiple steps are involved.(Fig. 5.1)

1. **Decoding:** Snort uses libcap to capture network traffic and feed it to the decoder. The decoder determines basic network characteristics for example source and destination addresses. It examines the different encapsulating protocols and checks them for sanity.
2. **Pre-processing:** Pre-processing reassembles packets to allow the extraction of the original application protocol data.
3. **Detection:** This the step where the rules are applied to the decoded and pre-processed packets. If a packet is conform with all rules it gets dropped, otherwise it gets examined deeper.
4. **Logging:** All information gathered about the network traffic is logged. A log can be a file, a terminal stream or a named pipe. The administrators can use the logs to send alarms and have deeper understanding how the attack went down.

5.2.1 Rules

Listing 5.1 shows an example snort rule. A Snort rule consists of action, protocol, source, direction, destination and body. We will only use the action alert. The action field can also be used to block traffic instead of alerting. The protocol field states for which protocols to look out. Examples are TCP, UDP , IP, ... The testbed utilizes Modbus/TCP thats why all our rules also use TCP. The next three fields declare where packets come from and where they go. They can contain both single and multiple addresses. "any" is used to check packets from all senders/receivers. The body can contain all kinds of information. Usually, it contains a message(msg) that is displayed in the logs and a signature ID (sid), which is used to identify every single rule.

```
alert tcp any any -> 192.168.1.1 80 (msg:""; content:""; sid:1;)
```

Listing 5.1: Example rule

5.3 Attack monitoring

For the detection we use Snort version 3.1.5.0 (from <https://snort.org/>). The configuration file was adapted to our network. The IP addresses of all docker containers, except the attacker, were written to \$HOME_NET. \$EXTERNAL_NET includes all address except our \$HOME_NET. The Modbus ports were also adapted accordingly, and the output has been set as alert_full and alert_json.

5.3.1 Monitoring of unauthenticated requests

Listing 5.2 shows a rule that allows us to detect unauthorized Modbus requests. It was taken from the Talos light speed snort community rules.[21b] The rule looks for all packets that are sent to one of the defined Modbus ports in our network from an unknown address. The flow keyword makes sure that the connection to the Modbus server was established by an external device. The Modbus preprocessor detects if its a read_coils request trough the modbus_func keyword. If all criteria are met an alert is triggered. The Talos ruleset also defines similar rules for all other Modbus functions.

```
alert tcp $EXTERNAL_NET any -> $HOME_NET [502,5020,5021,5022,5023]
  ( msg:"PROTOCOL-SCADA Modbus read coils from external source";
    flow:to_server,established; modbus_func:read_coils;
    classtype:protocol-command-decode; sid:17788; rev:5; )
```

Listing 5.2: Rule used to detect unauthorized read_coil requests

5.3.2 Monitoring of flooding attacks

Listing 5.3 is a rule that can detect SYN-Flood attacks. The keyword detection_filter triggers the alert when more than 70 packets arrive at a destination in 10 seconds. Those parameters have to be adjusted to the environment. The flag keyword makes sure that only SYN packets are tracked. It can also be omitted to send alerts for general flooding attacks.

```
alert tcp any any -> $HOME_NET [502,5020,5021,5022,5023]
(msg:"SYN Flood Detected"; sid:1000001; rev:1;
detection_filter:track by_dst, count 70, seconds 10;
flags: S;)
```

Listing 5.3: Rule used to detect SYN-Flood attacks

5.3.3 Monitoring of the multistage attack

To detect the multistage attack a combination of the Snort rules mentioned above and Snort inspectors can be used. In reconnaissance stage is a two step process. A fast IP/port scanner can be noticed through a flooding rule, since it will send a lot of packets in a short time. A more advanced way of detecting scans is the Snort port scan inspector it configured to detect “portscans” where an attacker scans multiple ports of one target or “portsweeps” an attacker scans a port on multiple targets. In the second stage of reconnaissance, the attack gathers information about the register. It uses read_requests which will be detected by the unauthorized request rule 5.2. A more advanced attack would passively tap the network traffic and draw from it information. This makes it undetectable for Snort since no packets are sent over the network. The same principle applies utilized through man in the middle. After the ARP-poisoning the attacker looks like it belongs to the network. We can also use the nature of the ARP-spoofing attack to detect it. If one IP address has multiple MAC-addresses associated to it. It is a clear hint that ARP-spoofing is happening. The Snort ARP Spoof Inspector is able to detect this.

6 Summary and Conclusion

Our first research question was: “How can we detect attack on an ICS through network analysis?”. In search of an answer we compared advantages and disadvantages of different network intrusion detection solutions. From there, we explored several ways to detect attacks using Snort. Our second research question was: “How to develop a demonstration of a Network Intrusion Detection System applied in the context of Industrial Control Systems?”. Therefore, we used an emulated PLC network testbed and extended it. An abstract process of cooking tea was implemented that takes advantage of a multiple PLCs that work together to produce one outcome. Different kinds of attacks were integrated to test the detection capabilities of our NIDS solution. The testbed, the attacks and the NIDS together make a functional demonstration of network intrusion detection applied to industrial control systems and show that NIDS can play a valuable part in the security of Industrial Control Systems.

6.1 Limitations

Our demonstration has some limitations. We used an emulated testbed which is very convenient for configuration and use. This solution also comes with some disadvantages. The emulated testbed runs on a PC with more computation power and storage than a commercial available PLC has. When flooding attacks and the PLCs are run on the same hardware without resource limitations they restrict themselves mutually, because storage and computation capacity are shared. It still works for our demonstration, since the high amount of network traffic can be measured and also detected. The detection we performed only concentrated on attacks that abused the SCADA protocol. An intrusion detection system can also be harmed by other means. For the intrusion detection we wanted to work with Snort3, due to its currentness and relevance in the future. This has created a few problems. It has sparse documentation and not a lot of experiences that can be found online. The configuration of the inspector used to detect ARP-poisoning is simple in theory. We still did not manage to produce the right alerts. There are no up-to-date solutions to this issue online and no help was given on snort forums. We still believe that with small tweaks to the configuration files the feature can be used to detect APR-poisoning attacks.

6.2 Future Work

As mentioned in the limitations above do our detection rules only concentrate on protocol based attacks. The real attack Stuxnet modified the programs running on the PLC. To get to those PLCs the attack also leaves traces in the network while it uploads the new program to the PLC. It would be interesting to search for ways to detect packets which include unwanted updates of a PLC program.

This thesis only worked with an emulated ICS. Real hardware often comes with new and unforeseen challenges. For example how to integrate the intrusion detection system in an existing system. The IDS has to be placed at strategic relevant locations to detect all import traffic. Therefore, a tool like Zeek may be more relevant, as it can collect data from different places through the cluster feature.

The testbed used in this thesis relied on Modbus for the communication in the industrial control system. Modbus is a simple protocol and has no real security features built in. There exist newer protocols, for example OPC Unified Architecture that are used in industrial control systems. Snort does not include any premade preprocessor for those programs, which makes it harder to create a reliable NIDS solution. The basic concepts of rules and inspectors are still transferable and open ways to apply network intrusion detection in Industrial Control Systems.

List of Figures

2.1	Example of an ICS	4
2.2	Modbus/TCP Packet	5
4.1	Ladder logic cook PLC	14
4.2	Ladder logic spice PLC	15
4.3	Ladder logic milk PLC	15
4.4	Ladder logic pump PLC	16
4.5	HMI view	17
4.6	TCP - Handshake	19
5.1	Snort detection flow	23

List of Tables

2.1	Modbus-Function Codes	6
2.2	Modbus Data Modeling	6
5.1	Comparison of different NIDS-Solutions	22

Bibliography

- [16] *Smod framework*. 2016. URL: <https://github.com/0x0mar/smod> (visited on 08/25/2021).
- [18a] *Metasploit Modbus Client Utility*. 2018. URL: <https://www.rapid7.com/db/modules/auxiliary/scanner/scada/modbusclient/> (visited on 08/25/2021).
- [18b] *Metasploit Modbus Version Scanner*. 2018. URL: <https://www.rapid7.com/db/modules/auxiliary/scanner/scada/modbusdetect/> (visited on 08/25/2021).
- [21a] *SWaT dataset*. 2021. URL: <https://itrust.sutd.edu.sg/itrust-labs-datasets/> (visited on 08/25/2021).
- [21b] *Talos*. 2021. URL: <https://snort.org/talos> (visited on 08/25/2021).
- [21c] *Zeek Cluster Setup*. 2021. URL: <https://docs.zeek.org/en/master/cluster-setup.html> (visited on 08/25/2021).
- [Emb21] V. Embacher. “Development of a Testbed to Demonstrate Attacks on Emulated PLC Networks.” In: (2021).
- [GT16] H. R. Ghaeini and N. O. Tippenhauer. “Hamids: Hierarchical monitoring intrusion detection system for industrial control systems.” In: *Proceedings of the 2nd ACM Workshop on Cyber-Physical Systems Security and Privacy*. 2016, pp. 103–111.
- [H+18] K. E. Hemsley, E. Fisher, et al. *History of industrial control system cyber incidents*. Tech. rep. Idaho National Lab.(INL), Idaho Falls, ID (United States), 2018.
- [Kas21] I. Kaspersky. “Threat landscape for industrial automation systems.” In: (2021).
- [KL14] E. D. Knapp and J. T. Langill. *Industrial Network Security: Securing critical infrastructure networks for smart grid, SCADA, and other Industrial Control Systems*. Syngress, 2014.
- [Kor+17] M. Korman, M. Vålja, G. Björkman, M. Ekstedt, A. Vernotte, and R. Lagerström. “Analyzing the effectiveness of attack countermeasures in a scada system.” In: *Proceedings of the 2nd Workshop on Cyber-Physical Security and Resilience in Smart Grids*. 2017, pp. 73–78.

Bibliography

- [S+11] K. Stouffer, J. Falco, K. Scarfone, et al. "Guide to industrial control systems (ICS) security." In: *NIST special publication* 800.82 (2011).
- [SM07] K. Scarfone and P. Mell. "Guide to Intrusion Detection and Prevention Systems (IDPS)." In: *NIST special publication* 800.94 (2007).
- [Sno21] Snort. "Snort 3 User Manual." In: (2021).